

NM

Shared Memory for Coding Agents

System Architecture Specification

Abstract

Coding agents in an organisation repeat the same project-specific mistake across sessions because they have no shared memory. NM is an MCP-served note graph that watches every coding session, distills the moments where the agent got stuck into a compact field of notes and injects the relevant notes into any future agent's context the moment it touches the same file. The system is named after the way we coordinated always-on agents that manage a Node.js managed stream of notes from a Shared Collection of scripts at Charles the Explorer that guards the graph and injections — Guarded by a filter of SQL to see, a Convex hosted SQL to read & write, a deployed reactive graph, a Verand an deployed Nimble dashboard. This document specifies the architecture of this its assigned specification platform, the data assigned to the execution platform, and the data flow between them.

Track and rubric

Submitted to Always-On Agents (sponsors: Nia + Tensorlake) at the Nozomio hackathon (May 9, 2026). Scoring weights: Background Execution 30%, Statefulness 25%, Agentic Depth 20%, Demo & Presentation 10%, Judge's Personal Rating 10%. The architecture is designed so that disabling background execution or the persistent note graph would cause the demo to fail — the rubric explicitly tests this.

Companion documents

- NM.md —Product specification, demo arc, statefulness story.
- SPEC.md —Sponsor-platform integration spec (deploy commands, env vars, file ownership).
- SCHEMA.md —Database schema reference for nm.db and Convex mirror.

1. System architecture

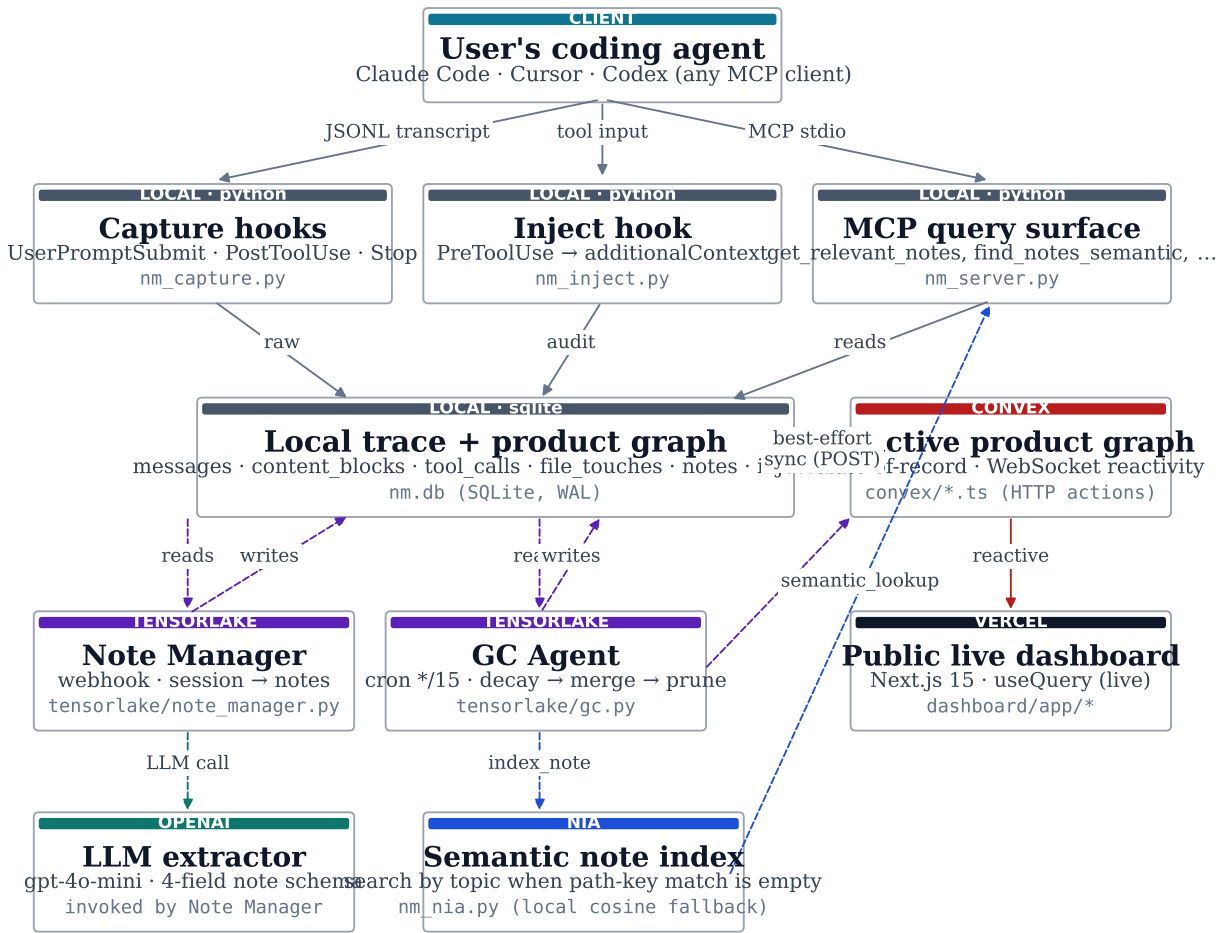
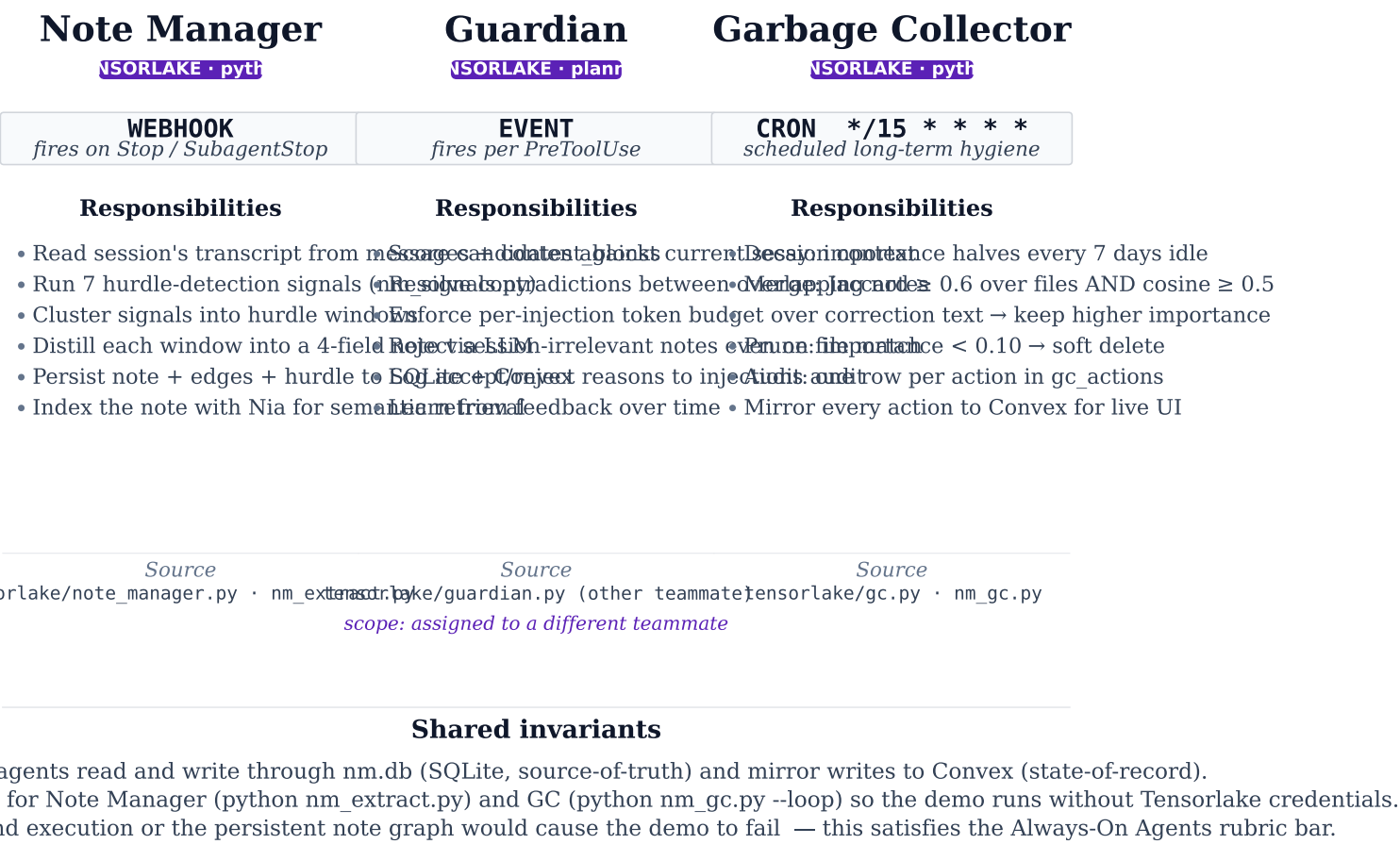


Figure 1. End-to-end data flow. Solid arrows are inline (in the latency-critical hook path). Dashed arrows are best-effort and fail-open.

2. Always-on agents



3. Data schema

The schema is partitioned into four modules, mirroring the OpenInference / OpenTelemetry GenAI semantic conventions: a verbatim trace, two derived projections for fast queries, the bipartite product graph, and an audit trail. The trace + projections live exclusively in local SQLite (latency-critical, append-heavy). The product graph and audit tables are mirrored to Convex for cross-machine reactivity.

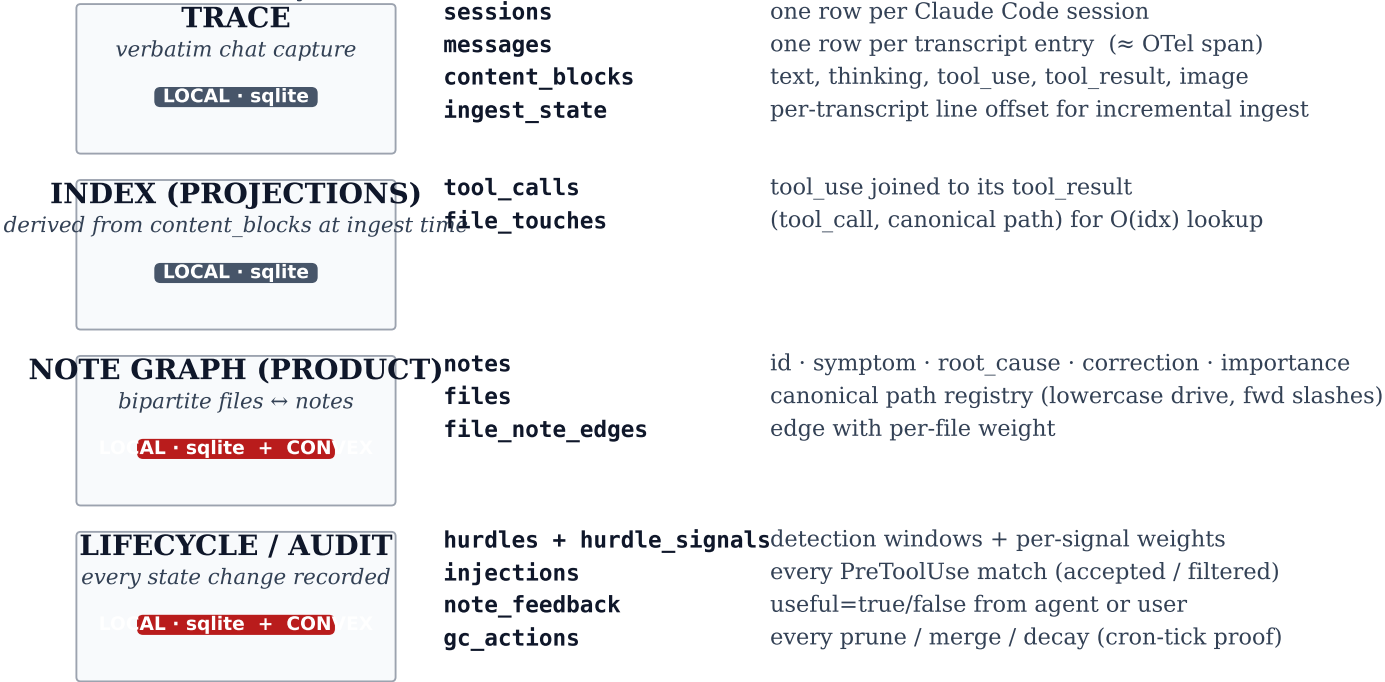


Figure 3. Database schema, organised into four modules. The full reference (with column types and indexes) is in SCHEMA.md.

4. Platform allocation

Each component is assigned to exactly one execution platform. SQLite stays the source of truth for write-side latency; Convex is the state-of-record for cross-machine reads; Tensorlake hosts the background agents; Vercel hosts the public dashboard URL; NIA and OpenAI provide auxiliary AI capabilities. Every sponsor integration has a local fallback — the demo runs end-to-end without any deploy.

Component	Role	Platform	Notes
nm_capture.py	transcript capture	OCAL · pytho	<i>inline, latency-critical (<50ms)</i>
nm_inject.py	PreToolUse hook + injection	OCAL · pytho	<i>inline, latency-critical</i>
nm_server.py	MCP query surface	OCAL · pytho	<i>stdio MCP; per coding-agent process</i>
nm.db	verbatim trace, projections, hooks	LOCAL · sqlite	<i>WAL, single-writer, fast</i>
Note Manager	session → notes via LLM	TENSORLAKE	<i>webhook on Stop; CLI fallback: nm_extract.py</i>
GC Agent	decay → merge → prune	TENSORLAKE	<i>cron */15 min; CLI fallback: nm_gc.py --loop</i>
Guardian Agent	per-injection filter	TENSORLAKE	<i>planned; assigned to other teammate</i>
Note graph + audit	reactive state-of-record	CONVEX	<i>mirrored from SQLite via HTTP actions</i>
Public dashboard	Next.js + useQuery (live)	VERCEL	<i>submission URL; reads only Convex</i>
Semantic note index	search by topic / fallback	NIA	<i>local cosine ranker if NIA_API_KEY unset</i>
LLM extractor	4-field note distillation	OPENAI	<i>gpt-4o-mini default, override via env vars</i>

Figure 4. Component-to-platform allocation. Sponsor platforms in italics each have a local fallback path.